SEMAINE 7

Autoencodeurs variationnels (VAE) et variantes

ELBO · KL · Reparamétrisation · β-VAE · CVAE

S7 — OBJECTIFS (CE QUE VOUS SAUREZ FAIRE À LA FIN)

1. Expliquer pourquoi l'autoencodeur classique ne suffit pas pour générer de nouvelles données.
2. Définir le modèle latent probabiliste d'un VAE : $p(z)$, $p_\theta(x \mid z)$ et $p_\theta(x)$.
3. Comprendre pourquoi la loi a posteriori $p_\theta(z \mid x)$ est difficile à calculer exactement.
4. Suivre la dérivation de la **ELBO** et interpréter ses deux termes.
5. Comprendre le rôle de la divergence **KL** et de l'**astuce de reparamétrisation**.
6. Expliquer le fonctionnement global d'un VAE, puis distinguer VAE standard, β-VAE et CVAE.
7. Comprendre comment un VAE permet la **génération**, l'**interpolation** et la **génération contrôlée**.

1 Fil conducteur : de S6 à S7

MISE EN CONTEXTE — DE LA RECONSTRUCTION À LA GÉNÉRATION

En S6, nous avons vu qu'un autoencodeur classique apprend à reconstruire une observation x à partir d'un code latent z. Cette idée est très utile pour compresser et représenter les données.

Mais une difficulté majeure subsiste : rien ne garantit que l'espace latent appris soit organisé de manière suffisamment régulière pour échantillonner de nouveaux points et générer de nouvelles observations plausibles. Cette semaine, nous allons donc ajouter une ***structure probabiliste*** *au latent afin de rendre la génération possible.*

QUESTION CENTRALE DE LA SEMAINE

Comment structurer l'espace latent pour pouvoir générer de nouvelles données de manière cohérente ?

LIEN AVEC LE PLAN DU COURS

La S6 a introduit les **autoencodeurs profonds** et leurs limites génératives. La S7 répond directement à cette limite en introduisant les **VAE**, puis deux variantes importantes : le **β-VAE** et le **CVAE**. Cela correspond exactement au contenu prévu dans le plan : **ELBO, KL, β-VAE, CVAE et génération contrôlée**.

2 Pourquoi l'autoencodeur classique ne suffit pas

2.1 Rappel rapide

Un autoencodeur classique apprend une chaîne du type

$$x \longrightarrow z \longrightarrow \tilde{x},$$

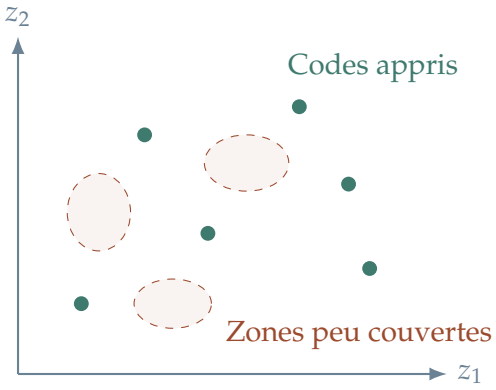
et optimise une erreur de reconstruction, par exemple

$$\mathcal{L}_{\mathrm{AE}}(x, \tilde{x}) = \|x - \tilde{x}\|^2.$$

LIMITATION FONDAMENTALE

L'autoencodeur classique apprend à bien reconstruire les données observées, mais il n'apprend pas explicitement une densité $p_\theta(x)$ ni une loi simple sur l'espace latent. On obtient donc souvent un espace latent **irrégulier** : certaines régions sont bien couvertes, d'autres beaucoup moins.

Avant de passer au VAE, retenez l'idée suivante : un bon autoencodeur n'est pas nécessairement un bon générateur. Il manque une hypothèse probabiliste sur les codes latents eux-mêmes.



Dans un autoencodeur classique, rien ne garantit qu'un point latent choisi au hasard soit pertinent.

CONSÉQUENCE PRATIQUE

Si l'on choisit un point z arbitrairement dans l'espace latent et qu'on le passe au décodeur, il est fréquent d'obtenir une sortie incohérente. Le décodeur n'a été entraîné qu'autour des codes produits par l'encodeur sur les données observées.

3 Le modèle génératif d'un VAE

3.1 Idée générale

Le VAE introduit un **modèle probabiliste à variable latente**. L'observation x n'est plus reliée à un seul code latent déterministe, mais à un mécanisme de génération en deux étapes :

1. on échantillonne un latent z selon une loi **a priori** $p(z)$;
2. on génère ensuite x selon une loi conditionnelle $p_\theta(x \mid z)$.

LES TROIS DISTRIBUTIONS CENTRALES DU VAE

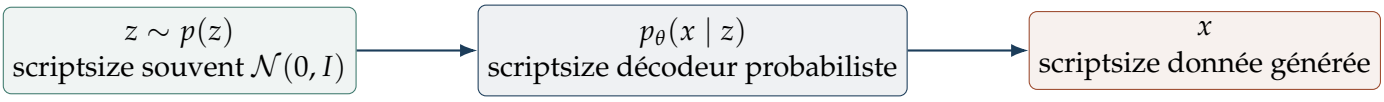
$$p(z), \qquad p_\theta(x \mid z), \qquad p_\theta(x) = \int p(z)\, p_\theta(x \mid z)\, dz.$$

— $p(z)$: la **loi a priori** du latent, choisie avant d'observer x, souvent $\mathcal{N}(0, I)$;

— $p_\theta(x \mid z)$: le **modèle génératif / décodeur probabiliste**, paramétré par θ ;

— $p_\theta(x)$: la **distribution marginale** des données obtenue en tenant compte de tous les z possibles.

Lecture des paramètres : θ désigne les paramètres du décodeur. Plus loin, ϕ désignera les paramètres de l'encodeur qui approxime la loi a posteriori.



LECTURE INTUITIVE

Le latent z joue le rôle d'une cause cachée simplifiée. Au lieu de mémoriser directement les observations, le modèle suppose que les données sont produites à partir de facteurs latents plus compacts.

3.2 Pourquoi ce modèle aide à générer

Choisir une loi a priori simple et continue, comme $p(z) = \mathcal{N}(0, I)$, donne au modèle une **référence commune** pour organiser les codes latents.

Le terme KL de l'ELBO pénalise, pour chaque observation x, une loi a posteriori approchée $q_\phi(z \mid x)$ trop éloignée de la loi a priori $p(z)$. Il encourage ainsi l'espace latent à rester compatible avec une distribution dont on sait facilement échantillonner.

Après apprentissage, la génération suit alors deux étapes simples :

$$z \sim p(z), \qquad x \sim p_\theta(x \mid z).$$

En pratique, le décodeur fournit les paramètres de $p_\theta(x \mid z)$; on peut utiliser sa moyenne ou échantillonner selon le type de sortie choisi.

4 Le problème d'inférence

4.1 La quantité recherchée

Lorsque l'on observe une donnée x, on aimerait connaître la distribution des latents plausibles qui ont pu la générer. Cette quantité est la loi a posteriori :

$$p_\theta(z \mid x) = \frac{p_\theta(x, z)}{p_\theta(x)} = \frac{p(z)\, p_\theta(x \mid z)}{p_\theta(x)}.$$

4.2 Pourquoi ce calcul est difficile

La difficulté vient du terme de normalisation au dénominateur :

$$p_\theta(x) = \int p(z)\, p_\theta(x \mid z)\, dz.$$

LE PROBLÈME D'INFÉRENCE *(problème → échec → solution)*

Problème : on voudrait calculer exactement $p_\theta(z \mid x)$.
Échec : le calcul de $p_\theta(x)$ nécessite une intégrale sur tous les latents possibles, et cette intégrale ne possède généralement pas de forme fermée simple lorsque $p_\theta(x \mid z)$ est représenté par un réseau de neurones.
Solution : approximer la loi a posteriori exacte par une distribution plus simple, notée $q_\phi(z \mid x)$.

LIEN AVEC LA S4

La S4 a introduit le **problème d'inférence** pour les variables latentes. La S7 en est le prolongement naturel : on ne se contente plus de dire que l'inférence est difficile, on construit maintenant une solution opérationnelle grâce à l'**inférence variationnelle**.

5 Approximer la loi a posteriori : inférence variationnelle

5.1 L'idée centrale

On introduit une famille de distributions approchées $q_\phi(z \mid x)$, paramétrée par ϕ. Cette distribution est produite par l'encodeur du VAE.

INTERPRÉTATION DE $q_\phi(z \mid x)$

Pour une observation donnée x, l'encodeur ne renvoie plus un seul point latent. Il renvoie les paramètres d'une **distribution** sur les latents plausibles. En pratique, on choisit souvent une gaussienne diagonale :

$$q_\phi(z \mid x) = \mathcal{N}\left(z\,;\, \mu_\phi(x), \mathrm{diag}\left(\sigma^2_\phi(x)\right)\right).$$

Autrement dit, l'encodeur produit une moyenne $\mu_\phi(x)$ et un vecteur d'écarts-types $\sigma_\phi(x)$. La notation $\mathrm{diag}(\sigma_\phi^2(x))$ signifie que l'on suppose ici une covariance diagonale : chaque dimension latente possède sa propre variance, sans terme de covariance explicite entre dimensions.

5.2 Mesurer l'écart entre q_ϕ et la vraie loi a posteriori

Pour comparer $q_\phi(z \mid x)$ à la loi a posteriori exacte $p_\theta(z \mid x)$, on utilise la divergence de Kullback–Leibler :

$$\mathrm{KL}\big(q_\phi(z \mid x) \,\|\, p_\theta(z \mid x)\big) = \int q_\phi(z \mid x) \log \frac{q_\phi(z \mid x)}{p_\theta(z \mid x)}\, dz.$$

POINT À RETENIR

La divergence KL n'est pas une distance symétrique. Ici, elle mesure à quel point la distribution approchée $q_\phi(z \mid x)$ s'écarte de la loi a posteriori exacte $p_\theta(z \mid x)$.

CE QUI EST CALCULABLE ET CE QUI NE L'EST PAS

Pour une observation x, le VAE peut :

— calculer les paramètres de $q_\phi(z \mid x)$ avec l'encodeur ;

— échantillonner un z grâce à la reparamétrisation ;

— calculer $p_\theta(x \mid z)$ avec le décodeur ;

— calculer analytiquement la KL vers la loi a priori dans le cas gaussien standard.

En revanche, il ne sait généralement pas calculer directement :

$$p_\theta(x) = \int p_\theta(x \mid z) p(z)\, dz,$$

ni la loi a posteriori exacte $p_\theta(z \mid x)$.

La ELBO construit précisément un objectif exploitable malgré cette difficulté.

6 Dérivation pas à pas de la ELBO

6.1 Pourquoi introduire la ELBO ?

L'objectif probabiliste naturel serait de maximiser $\log p_\theta(x)$ pour les données observées. Mais $p_\theta(x)$ contient l'intégrale sur z vue précédemment, et la loi a posteriori exacte $p_\theta(z \mid x)$ est généralement difficile à calculer. On cherche donc une quantité **calculable** qui reste reliée à la log-vraisemblance : la ELBO.

AVANT LA DÉRIVATION : QUE FAUT-IL SUIVRE ?

La dérivation ne crée pas une nouvelle probabilité. Elle réécrit simplement la divergence KL entre l'approximation $q_\phi(z \mid x)$ et la loi a posteriori exacte. Le but est de faire apparaître $\log p_\theta(x)$ puis de l'isoler. À chaque étape, gardez donc en tête la question : comment remplacer une quantité difficile par des termes que l'on sait calculer ?

6.2 Étape 1 — partir de la KL

On part de :

$$\mathrm{KL}\big(q_\phi(z \mid x) \,\|\, p_\theta(z \mid x)\big) = \mathbb{E}_{q_\phi}[\log q_\phi(z \mid x)] - \mathbb{E}_{q_\phi}[\log p_\theta(z \mid x)].$$

Ici, $\mathbb{E}_{q_\phi}[\cdot]$ signifie : **prendre la moyenne lorsque z est tiré selon $q_\phi(z \mid x)$**. Cette notation compacte évite d'écrire une intégrale à chaque ligne.

6.3 Étape 2 — utiliser Bayes

Comme :

$$\log p_\theta(z \mid x) = \log p(z) + \log p_\theta(x \mid z) - \log p_\theta(x),$$

on remplace ce terme dans l'expression précédente.

6.4 Étape 3 — distribuer et regrouper

En substituant l'expression de Bayes et en distribuant l'espérance sur les trois termes, on obtient :

$$\mathrm{KL}\big(q_\phi(z \mid x) \,\|\, p_\theta(z \mid x)\big) = \mathbb{E}_{q_\phi}[\log q_\phi(z \mid x)] - \mathbb{E}_{q_\phi}[\log p(z)] - \mathbb{E}_{q_\phi}[\log p_\theta(x \mid z)] + \mathbb{E}_{q_\phi}[\log p_\theta(x)].$$

POURQUOI $\mathbb{E}_{q_\phi}[\log p_\theta(x)] = \log p_\theta(x)$?

Le terme $\log p_\theta(x)$ ne dépend pas de z : c'est une constante du point de vue de l'espérance sous $q_\phi(z \mid x)$. L'espérance d'une constante est cette constante elle-même, donc ce terme sort simplement de l'espérance sans être transformé.

On regroupe alors les deux premiers termes, qui forment exactement la définition de la KL entre $q_\phi(z \mid x)$ et la loi a priori $p(z)$:

$$\underbrace{\mathbb{E}_{q_\phi}[\log q_\phi(z \mid x)] - \mathbb{E}_{q_\phi}[\log p(z)]}_{=\mathrm{KL}(q_\phi(z|x) \,\|\, p(z))},$$

d'où l'expression intermédiaire :

$$\mathrm{KL}\big(q_\phi(z \mid x) \,\|\, p_\theta(z \mid x)\big) = \mathrm{KL}\big(q_\phi(z \mid x) \,\|\, p(z)\big) - \mathbb{E}_{q_\phi}[\log p_\theta(x \mid z)] + \log p_\theta(x).$$

BILAN DE L'ÉTAPE 3

Nous avons fait apparaître deux quantités que l'on peut traiter en pratique :

$$\mathrm{KL}(q_\phi(z \mid x) \,\|\, p(z)) \qquad \text{et} \qquad \mathbb{E}_{q_\phi}[\log p_\theta(x \mid z)].$$

Dans le VAE gaussien standard, la KL possède une formule analytique et l'espérance de reconstruction est estimée par échantillonnage. Il ne reste plus qu'à isoler $\log p_\theta(x)$.

6.5 Étape 4 — isoler $\log p_\theta(x)$

En passant $\log p_\theta(x)$ seul d'un côté de l'égalité, on obtient :

$$\log p_\theta(x) = \underbrace{\mathbb{E}_{z\sim q_\phi(z|x)}[\log p_\theta(x \mid z)] - \mathrm{KL}\big(q_\phi(z \mid x) \,\|\, p(z)\big)}_{\mathrm{ELBO}(x;\theta,\phi)} + \mathrm{KL}\big(q_\phi(z \mid x) \,\|\, p_\theta(z \mid x)\big).$$

DÉFINITION DE LA ELBO

$$\boxed{\mathrm{ELBO}(x;\theta,\phi) = \mathbb{E}_{z\sim q_\phi(z|x)}[\log p_\theta(x \mid z)] - \mathrm{KL}\big(q_\phi(z \mid x) \,\|\, p(z)\big)}$$

6.6 Étape 5 — interpréter la borne

Comme toute divergence KL est **non négative**,

$$\mathrm{KL}\big(q_\phi(z \mid x) \,\|\, p_\theta(z \mid x)\big) \geq 0,$$

on a automatiquement :

$$\log p_\theta(x) \geq \mathrm{ELBO}(x;\theta,\phi).$$

C'est pour cela qu'on parle de **borne inférieure de l'évidence**. En maximisant la ELBO, on maximise une borne inférieure de la log-vraisemblance.

LECTURE DES DEUX TERMES

— **Terme de reconstruction :**

$$\mathbb{E}_{q_\phi}[\log p_\theta(x \mid z)]$$

mesure la qualité de reconstruction du décodeur.

— **Terme de régularisation :**

$$\mathrm{KL}(q_\phi(z \mid x) \,\|\, p(z))$$

pénalise une loi a posteriori approchée trop éloignée de la loi a priori.

MESSAGE CLÉ

Maximiser la ELBO = bien reconstruire les données + garder un espace latent compatible avec la loi a priori.

MINI-EXEMPLE NUMÉRIQUE — LIRE LA PERTE VAE

Supposons que, pour une observation, la **perte de reconstruction** vaille 20 et que le terme de **KL** vaille 3. La perte négative de l'ELBO vaut alors :

$$\mathcal{L}_{\mathrm{VAE}} = 20 + 3 = 23.$$

Si l'on améliore fortement la reconstruction mais que les codes s'éloignent trop de la loi a priori, la KL augmente. Inversement, imposer une KL trop forte peut dégrader la reconstruction. Le VAE cherche donc un **compromis** entre fidélité des données et organisation du latent.

7 Cas gaussien : formule pratique de la KL

Dans le VAE standard, on choisit souvent :

$$p(z) = \mathcal{N}(0, I) \qquad \text{et} \qquad q_\phi(z \mid x) = \mathcal{N}\Big(z\,;\, \mu_\phi(x), \mathrm{diag}\big(\sigma_\phi^2(x)\big)\Big)\,.$$

Dans ce cas, la divergence KL se calcule analytiquement :

$$\mathrm{KL}\big(q_\phi(z \mid x) \,\|\, p(z)\big) = \frac{1}{2}\sum_{j=1}^{d}\left(\mu_{\phi,j}(x)^2 + \sigma_{\phi,j}(x)^2 - \log \sigma_{\phi,j}(x)^2 - 1\right).$$

REMARQUE

Nous admettons ce résultat classique sans le démontrer ici : il s'obtient en développant explicitement la définition de la KL entre deux gaussiennes, l'une diagonale et l'autre standard, puis en intégrant terme à terme. Ce n'est pas une étape manquée : c'est un calcul standard que nous choisissons de ne pas refaire dans ce cours, afin de nous concentrer sur son interprétation.

INTERPRÉTATION

Cette expression pénalise les moyennes qui s'éloignent de 0 et les variances qui s'éloignent de 1. Elle pousse donc les latents appris à rester compatibles avec la loi a priori gaussienne standard.

DU CONCEPT AU CODE — CALCUL DE LA KL

Dans une implémentation, on produit souvent $\log \sigma_\phi^2(x)$, noté `logvar`, plutôt que $\sigma_\phi(x)$ directement. La KL s'écrit alors sous une forme compacte :

```
kl = -0.5 * torch.sum(1 + logvar - mu.pow(2) - logvar.exp())
```

À observer : ce fragment n'est pas une nouvelle théorie ; il traduit directement la formule analytique précédente.

8 L'astuce de reparamétrisation

8.1 Pourquoi en a-t-on besoin ?

Pour apprendre le VAE par descente de gradient, il faut pouvoir rétropropager à travers l'échantillonnage latent. Or, un tirage aléatoire direct :

$$z \sim q_\phi(z \mid x)$$

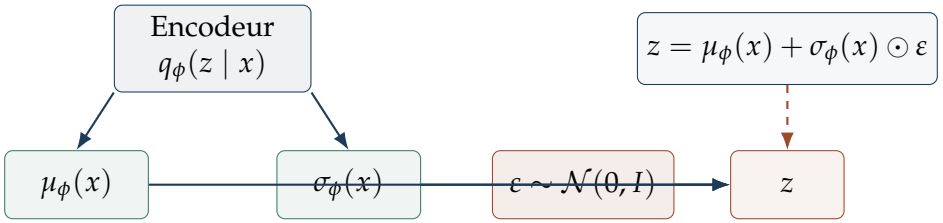
ne fournit pas une dépendance différentiable simple entre le résultat échantillonné et les paramètres ϕ de l'encodeur.

8.2 L'idée

On réécrit le tirage sous la forme :

$$z = \mu_\phi(x) + \sigma_\phi(x) \odot \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, I).$$

Le bruit aléatoire est isolé dans ε, tandis que $\mu_\phi(x)$ et $\sigma_\phi(x)$ restent des fonctions déterministes de l'encodeur.



POURQUOI CELA FONCTIONNE

Le bruit est maintenant séparé de la partie apprise. On peut donc rétropropager à travers $\mu_\phi(x)$ et $\sigma_\phi(x)$ par des opérations standards, tout en conservant l'effet stochastique voulu.

DU CONCEPT AU CODE — ÉCHANTILLONNAGE REPARAMÉTRÉ

```
std = torch.exp(0.5 * logvar)
eps = torch.randn_like(std)
z = mu + std * eps
```

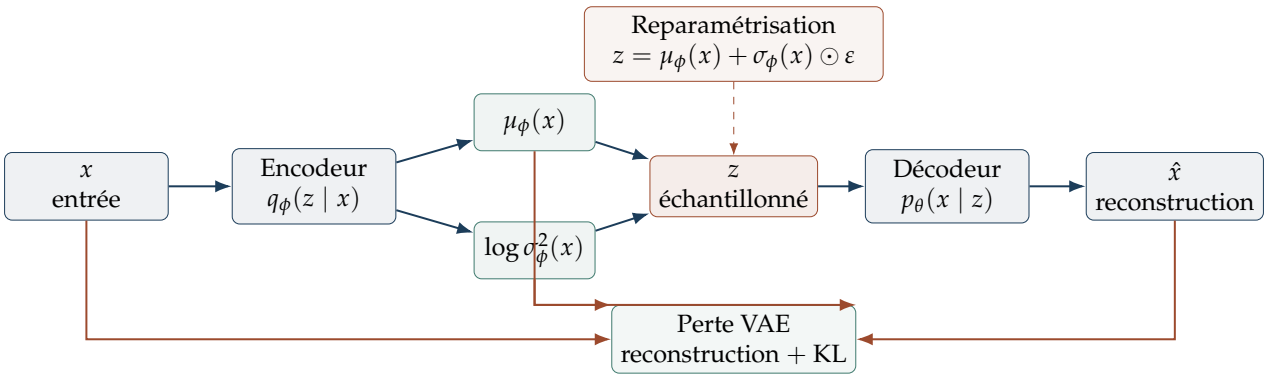
À observer : `eps` fournit l'aléa ; `mu` et `std` restent différentiables.

9 Architecture complète d'un VAE

Le VAE comporte deux réseaux :

— un **encodeur** qui produit $\mu_\phi(x)$ et $\log \sigma^2_\phi(x)$;

— un **décodeur** qui produit la distribution de reconstruction $p_\theta(x \mid z)$.



Le VAE combine un encodeur probabiliste, un échantillonnage reparamétré et un décodeur ; l'apprentissage équilibre reconstruction et régularisation KL.

10 Fonction de perte et entraînement

10.1 La perte VAE

En pratique, on maximise la ELBO, ce qui revient à minimiser sa négative :

$$\mathcal{L}_{\text{VAE}}(x;\theta,\phi) = -\mathbb{E}_{z\sim q_\phi(z|x)}[\log p_\theta(x \mid z)] + \text{KL}\big(q_\phi(z \mid x) \parallel p(z)\big).$$

10.2 Comment lire cette formule ?

— Le premier terme,

$$-\mathbb{E}_q[\log p_\theta(x \mid z)],$$

est le **coût de reconstruction probabiliste**. Selon le modèle de sortie choisi, il peut conduire à une BCE pour une sortie Bernoulli ou, à une constante près sous une gaussienne à variance fixée, à une MSE.

— Le second terme,

$$\text{KL}(q_\phi(z \mid x) \parallel p(z)),$$

est une **régularisation probabiliste** de l'espace latent.

Les paramètres θ du décodeur et ϕ de l'encodeur sont optimisés **ensemble** en minimisant cette perte sur les observations du mini-lot.

DU CONCEPT AU CODE — CALCUL DE LA PERTE

```
recon_loss = F.mse_loss(x_hat, x, reduction="sum")
kl_loss = -0.5 * torch.sum(1 + logvar - mu.pow(2) - logvar.exp())
loss = recon_loss + kl_loss
```

À observer : la `loss` totale est simplement la somme des deux idées théoriques vues plus haut : reconstruction + KL.

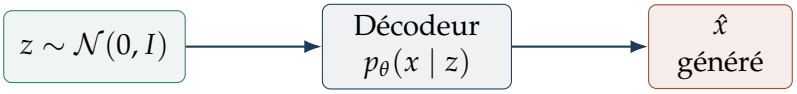
Note : en pratique, les implémentations utilisent parfois une somme (`sum`) ou une moyenne (`mean`) sur les pixels et le mini-lot. Ce choix modifie l'échelle numérique relative des deux termes ; il faut donc les normaliser de façon cohérente.

11 Génération et interpolation avec un VAE

11.1 Génération après entraînement

Une fois le VAE appris, la génération devient simple :

1. échantillonner un latent $z \sim p(z) = \mathcal{N}(0, I)$;
2. passer ce latent dans le décodeur ;
3. récupérer la nouvelle donnée générée.



DU CONCEPT AU CODE — GÉNÉRATION

```
z = torch.randn(n_samples, latent_dim)
x_params = decoder(z)
```

À observer : on n'utilise plus l'encodeur. On tire directement z dans la loi a priori puis le décodeur produit les paramètres de la distribution de sortie. Selon le modèle choisi, on utilise ensuite leur moyenne ou on échantillonne x.

11.2 Interpolation latente

On peut prendre deux observations x_1 et x_2 et représenter chacune par la moyenne de sa loi a posteriori approchée, par exemple :

$$z_1 = \mu_\phi(x_1), \qquad z_2 = \mu_\phi(x_2).$$

On interpole ensuite :

$$z_\alpha = (1 - \alpha)z_1 + \alpha z_2, \qquad \alpha \in [0, 1].$$

Utiliser les moyennes rend la visualisation reproductible. Si l'espace latent est bien structuré, le décodage de z_α produit généralement une transition progressive entre les deux observations.

12 Variante 1 : le β-VAE

12.1 Idée générale

Le β-VAE modifie la balance entre reconstruction et régularisation en introduisant un coefficient β devant la divergence KL :

$$\mathcal{L}_{\beta\text{-VAE}}(x) = -\mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x \mid z)] + \beta\, \mathrm{KL}\big(q_\phi(z \mid x) \,\|\, p(z)\big).$$

INTERPRÉTATION DE β

— $\beta = 1$: on retrouve le VAE standard ;

— $\beta > 1$: on renforce la contrainte sur le latent, ce qui peut favoriser des représentations plus **désentrelacées** ;

— $\beta < 1$: on privilégie davantage la reconstruction, au prix d'un latent moins régulier.

COMPROMIS IMPORTANT

Augmenter β renforce la pression vers la loi a priori et peut favoriser des facteurs latents plus séparés, mais le désentrelacement n'est **pas garanti**. Une valeur trop grande peut aussi détériorer la reconstruction. Il faut donc ajuster β selon l'objectif poursuivi.

13 Variante 2 : le CVAE

13.1 Pourquoi conditionner ?

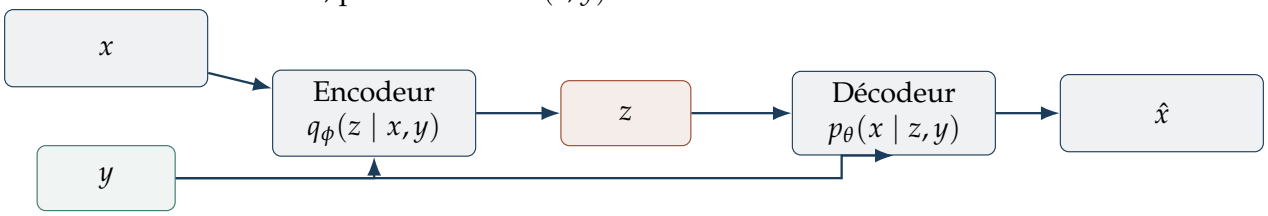
Le **Conditional VAE** (CVAE) ajoute une variable d'information y pour contrôler la génération. Par exemple, y peut représenter la classe d'un chiffre MNIST, un style, une étiquette sémantique, etc.

13.2 Formes conditionnelles

Dans un CVAE, on utilise typiquement :

$$q_\phi(z \mid x, y), \qquad p_\theta(x \mid z, y), \qquad \text{et souvent encore } p(z) = \mathcal{N}(0, I).$$

L'encodeur et le décodeur reçoivent donc tous deux l'information y pendant l'apprentissage. Pour générer, on choisit une condition y, on tire z dans la loi a priori, ou dans une loi a priori conditionnelle si le modèle en utilise une, puis on décode (z, y).



GÉNÉRATION CONTRÔLÉE

Si l'on fixe une valeur de y (par exemple la classe « 7 »), le CVAE peut générer des échantillons cohérents avec cette contrainte. On ne demande plus seulement *générer quelque chose*, mais *générer quelque chose de la classe désirée*.

DU CONCEPT AU CODE — GÉNÉRATION CONDITIONNELLE

```
z = torch.randn(n_samples, latent_dim)
y = one_hot(labels)
x_new = decoder(z, y)
```

14 Comparaison synthétique des modèles

Modèle	Idée clé	Espace latent	Usage typique
Autoencodeur classique	Reconstruction seulement	Peu structuré	Compression, débruitage
VAE standard	Reconstruction + régularisation KL	Continu, régularisé par une loi a priori	Génération, interpolation
β-VAE	KL pondérée par β	Régularisation ajustable	Désentrelacement potentiel
CVAE	Génération conditionnée par y	Structuré et contrôlable	Génération contrôlée

15 Synthèse de la semaine 7 et transition vers S8

SYNTHÈSE — CE QU'IL FAUT RETENIR

— le VAE ajoute une **structure probabiliste** à l'autoencodeur ;
— il repose sur un modèle latent $p(z)$ et un décodeur probabiliste $p_\theta(x \mid z)$;
— la loi a posteriori exacte $p_\theta(z \mid x)$ est remplacée par une approximation $q_\phi(z \mid x)$;
— la **ELBO** équilibre **reconstruction** et **régularisation KL** ;
— l'**astuce de reparamétrisation** rend l'apprentissage différentiable ;

— le β-VAE renforce la contrainte sur le latent ;
— le CVAE permet la **génération contrôlée**.

AUTO-VÉRIFICATION

À la fin de cette semaine, vous devriez pouvoir :

1. expliquer pourquoi un VAE est génératif alors qu'un autoencodeur classique ne l'est pas directement ;
2. nommer les rôles de $p(z)$, $p_\theta(x \mid z)$ et $q_\phi(z \mid x)$;
3. expliquer pourquoi l'ELBO est une borne inférieure de $\log p_\theta(x)$;
4. interpréter les deux termes de la perte VAE ;
5. expliquer en une phrase l'astuce de reparamétrisation ;
6. distinguer VAE, β-VAE et CVAE.

TRANSITION VERS LA S8

"Les VAE définissent un modèle génératif explicite :

$$p_\theta(x) = \int p_\theta(x \mid z) p(z)\, dz,$$

mais cette vraisemblance marginale est généralement difficile à calculer exactement. Ils sont donc entraînés à l'aide d'une borne, l'ELBO.

*La S8 introduira une autre famille de modèles génératifs explicites : les **flots normalisants**. Grâce à des transformations inversibles et à la formule du **changement de variable**, ils permettent de calculer exactement la log-vraisemblance dans les architectures appropriées."*